

Exame Modelo 3 — Paradigmas de Programação — Época Normal / Recurso 2025/2026

Instituição	P.PORTO — Escola Superior de Tecnologia e Gestão
Tipo de Prova	Exame Escrito — Época Normal / Recurso
Curso	Licenciatura em Engenharia Informática / Licenciatura em Segurança Informática em Redes de Computadores
Unidade Curricular	Paradigmas de Programação
Ano Letivo	2025/2026
Duração	2 horas

Observações

- Não é permitida a consulta.
 - É estritamente proibida a utilização de coleções da biblioteca `java.util`.
-

Parte 1 (6 Valores)

Pergunta 1 (1,5 valores)

Explique as diferenças entre composição e herança em Java, descrevendo quando cada mecanismo é mais adequado. Justifique por que razão a composição é frequentemente preferida em relação à herança no desenvolvimento de software e forneça um exemplo prático que demonstre a composição entre duas classes.

Pergunta 2 (1,5 valores)

Explique detalhadamente o conceito de tipos enumerados (`enum`) em Java. Descreva as vantagens de utilizar `enum` em vez de constantes inteiras ou Strings para representar conjuntos fixos de valores. Demonstre como um `enum` pode ter atributos, construtores e métodos. Ilustre com um exemplo prático.

Pergunta 3 (1,5 valores)

O que são classes Wrapper em Java e qual o seu propósito? Explique os conceitos de Autoboxing e Unboxing e discuta um problema comum de desempenho em ciclos e um risco de comparação de valores (`==` vs `equals()`) com instâncias de `Double` ou `Integer` .

Pergunta 4 (1,5 valores)

A biblioteca de Input/Output (I/O) do Java divide-se principalmente em classes orientadas a bytes (Streams) e classes orientadas a caracteres (Readers/Writers). Explique a diferença fundamental entre estas duas famílias de classes e indique em que cenários práticos é adequada a utilização de cada uma delas.

Parte 2 (14 Valores)

1. Considere as seguintes interfaces `Vehicle` e `Route` . A interface `Route` define o contrato de uma rota de recolha associada a um veículo.

```
public interface Route {
    Vehicle getVehicle();
    void addAidBox(AidBox aidBox) throws RouteException;
    AidBox removeAidBox(AidBox aidBox) throws RouteException;
    AidBox[] getRoute();
    boolean equals(Object obj);
}
```

```
public interface Vehicle {
    String getCode();
    ItemType getSupplyType();
    double getMaxCapacity();
}
```

Pergunta 1a (3 valores)

Implemente a interface `Route` numa classe denominada `RouteImpl` . A rota deve ser associada a um veículo no momento da sua criação e conter um array de `AidBox` com capacidade máxima de 10 posições. O método `addAidBox` deve lançar `RouteException` se a caixa for nula, se a capacidade for atingida ou se a caixa já existir na rota. O método `removeAidBox` deve lançar `RouteException` se a caixa não for encontrada. Duas rotas são iguais se possuírem o mesmo veículo.

Pergunta 1b (2 valores)

Desenvolva o código necessário para testar a classe `RouteImpl` num método `main` .

2. Considere a interface `CollectionManager` :

```
public interface CollectionManager {  
    double getTotalCollectedByType(IInstitution inst, ItemType type);  
}
```

Pergunta 2a (4 valores)

Implemente os seguintes métodos na classe `CollectionManagerImpl` :

```
double getContainerLoad(Container container);
```

- Devolve o valor da última medição do contentor. Se for nulo ou sem medições, devolve 0.0.

```
boolean isContainerFull(Container container, double threshold);
```

- Devolve `true` se a última medição for superior à percentagem `threshold` da capacidade do contentor.

Pergunta 2b (5 valores)

Na classe `CollectionManagerImpl`, implemente o método `getTotalCollectedByType`, somando a carga de todos os contentores do tipo especificado que estejam com ocupação superior a 75% (`threshold = 0.75`).

Excertos de Código Fornecidos

```
public class InstitutionImpl implements IInstitution {  
    private AidBox[] aidBoxes;  
    private int numberOfAidBoxes;  
  
    public AidBox[] getAidBoxes() { return this.aidBoxes; }  
}
```